

Abstract State Machines (parte 1): stato ASM



Elvinia Riccobene
elvinia.riccobene@unimi.it

Abstract State Machines

- ASM = FSM con stati generalizzati
 - Le ASM rappresentano la forma matematica di Machine Virtuali
 - Estendono la nozione di Finite State Machine ampliando la definizione di **stato** e modificando la forma delle **transizioni**
 - **Esempio:** un orologio digitale che segna l'ora e viene aggiornato ad ogni *delta* intervallo di tempo

Abstract State Machines

s2

CurrTime: Real

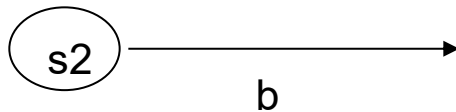
DisplayTime: Real

Delta: Real

+ : Real x Real -> Real

- **Stati di controllo** non strutturati sostituiti da stati (strutturati) che modellano:
 - dati complessi arbitrari (con domini di base e funzioni per la struttura) ed
 - operazioni per la manipolazione di dati
- **Stato**: multi-sorted first-order structures o *algebra*

Abstract State Machines



- Le **transizioni** sono “regole” che descrivono il cambiamento di funzioni da uno stato al successivo
 - basic **transition** rule: “**if** *Condition* **then** *Updates*”

Abstract State Machines

- Diversi, ma finiti e semplici costruttori di regole:
 - parallelismo (**par**) ed azioni sequenziali (**seq**)
 - iterazioni (**while**) ed invocazione di sottomacchine
 - non-determinismo (**choose**) e parallelismo sincrono non limitato (**forall**)
 - multi-agenti sincroni/asincroni

Abstract State Machines

- Dotate di un ambiente di tool per
 - Editing
 - Simulazione
 - Validazione
 - Verifica (via model checking)
 - Generazione di casi di test
- Un modello ASM può essere visto come pseudocodice su ADT (strutture dati astratte)

Abstract State Machines

- Tool e altro materiale disponibile sul sito di **Asmeta** (ASM mETAmodeling)

- <https://asmeta.github.io/>

Asmeta	Overview
Overview	This site is dedicated to the Abstract State Machine Metamodel (AsmM, in brief) , a metamodel for the Abstract State Machines (ASMs) formal method developed by following the guidelines of the Model-Driven Engineering (MDE) .
User Guide	We exploit the metamodeling approach suggested by MDE to achieve the goals of developing a unique notation for ASM, a notation independent from any specific implementation syntax and allowing a more direct encoding of the ASM mathematical concepts and constructs, and tackling the problem of ASM tool interoperability.
AsmM: the ASM metamodel	The AsmM can be seen as a <i>pivot metamodel</i> towards a definition of a standard family of languages for the ASMs and a systematic integration of ASM tools. Through metamodels and model transformations, the metamodel of each tool (usually not given explicitly, but is built-in into the tool that provide it) is linked to those of other tools by the logical pivot metamodel which abstracts a certain number of general concepts about ASMs.
AsmetaL	The AsmM is defined by using EMF , the modelling framework proposed by
Asmeta Tools	
AsmetaL compiler	
Asmeta Simulator	
Asmeta Validator	
Asmee eclipse plugin	
Papers	
Web Resources	
Developer Information	
Source Repository	
Architecture	
Open Issues	
Support	
People	

Abstract State Machines

Domini:

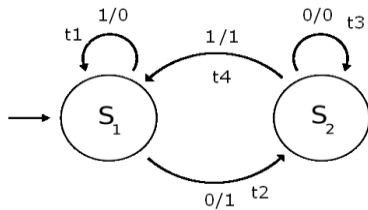
State: insieme degli stati

Funzioni:

ctl_State: State

input: String

output: String



Regole di transizione:

r_{s1_1} = if ctl_State = s1 and input = "1" then

 ctl_State := "s1"

 output := "o"

r_{s1_0} = if ctl_State = "s1" and input = "o" then ...

r_{s2_1} = ...

r_{s2_0} = ...

Abstract State Machines

- ASM = (header, body, main rule, initialization)

Domini:

State: insieme degli stati

Funzioni:

ctl_State: State

input: String

output: String

Regole di transizione:

r_s1_1 = if ctl_State = s1 and
input = "1"

then

ctl_State := "s1"

output := "o"

r_s1_0 =

r_s2_1 = ...

r_s2_0 = ...

main rule

initialization

asm FSM

import StandardLibrary

signature:

controlled ctl_State: String

monitored input: String

out output: String

header

definitions:

rule r_s1_1 = if ctl_ = "s1" and input = "1" then

par ctl_State := "s1", output := "o" endpar

endif

rule r_s1_0 = if ctl_State = "s1" and input = "o" then ...

rule r_s2_1 = ...

rule r_s2_0 = ...

body

main rule r_Main = par r_s1_1, r_s1_0, r_s2_1, r_s2_0 endpar

default init so:

function currentState = "s1"

Da FSM a ASM (1)

- Le ASM sono analoghe alle FSM
- Le differenze riguardano
 - la **concezione degli stati**:
 - nelle FSM esiste un unico stato di controllo (ctl_state), che può assumere valori in un insieme finito di un certo tipo
 - nelle ASM lo stato è più complesso (*algebra*)
 - le **condizioni di input e le azioni di output**
 - nelle FSM alfabeto finito
 - nelle ASM: input qualsiasi espressione, azioni generali

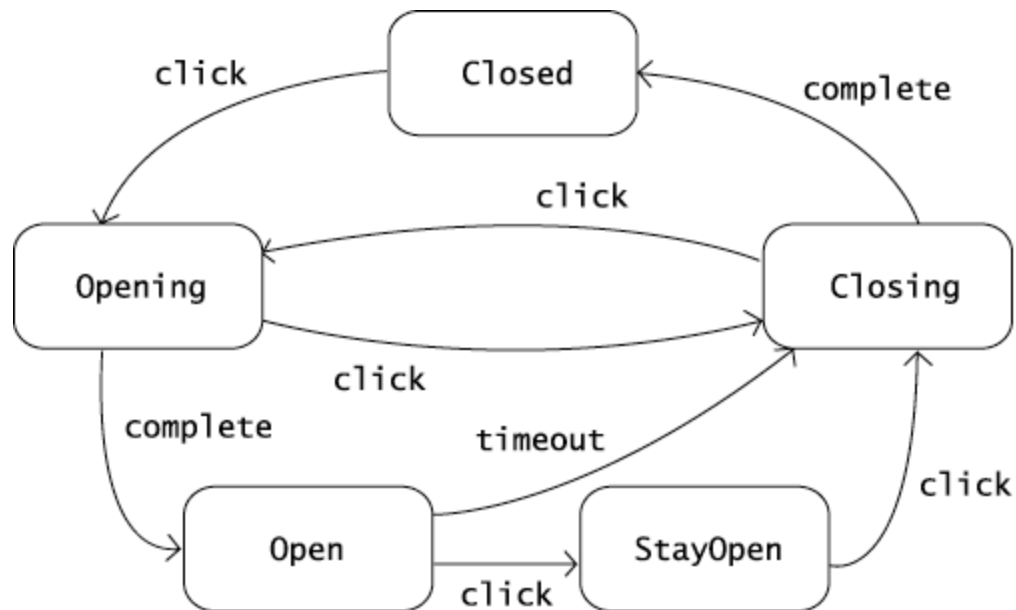
AsmetaL

- Le ASM hanno una notazione matematica in termini di algebra
- Useremo AsmetaL come notazione concreta per l'editing di modelli ASM

Esempio

Una porta girevole opera in modalità singolo bottone. Se la porta è chiusa, premendo il bottone la porta inizia ad aprirsi. Se si preme ancora una volta il bottone prima che la porta si apre del tutto, la porta inizia a chiudersi. Se, invece, permettiamo alla porta di aprirsi del tutto, una volta aperta la porta inizia a chiudersi automaticamente dopo un timeout di 2-secondi. Per impedire la chiusura automatica e mantenere la porta aperta occorre premere il bottone; a questo punto, un'ulteriore pressione del bottone, fa sì che la porta inizi a chiudersi. Se si preme ancora una volta il bottone prima che la porta si chiuda del tutto, la porta inizia ad aprirsi.

Modello FSM dell'esempio



Modello ASM dell' esempio

Domini:

Stati: insieme degli stati

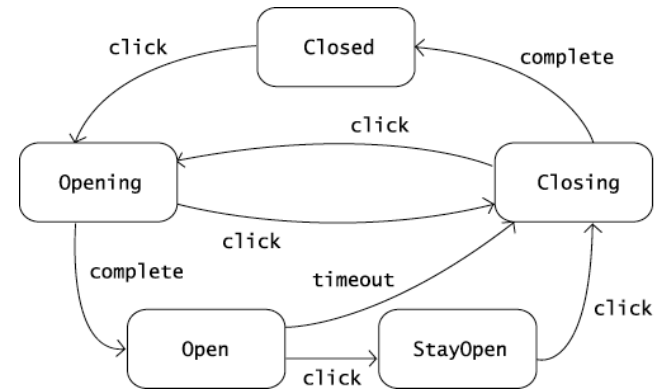
Funzioni:

ctl_state: Stati

click: boolean

complete: boolean

timeout: boolean



Regole di transizione:

if ctl_state = Opening and click
then
 ctl_state := Closing

if ctl_state = Closing and click
then
 ctl_state := Opening

if ctl_state = Closing and complete
then
 ctl_state := Closed

.....

Inizializzazione:

State = {Opening, Closing, Open,}
ctl_state = Open

Ancora un esempio: vending machine (1)

Una macchinetta automatica dispensa caffè' o latte. La macchinetta accetta solo monete da 50 centesimi e da 1 euro. Se viene inserita una moneta da 50 centesimi, la macchinetta dispensa latte (se disponibile); se viene inserita una moneta da 1 euro, la macchinetta dispensa caffè' (se disponibile).

Se viene dispensata una bevanda, la sua disponibilità viene decrementata e la moneta viene conservata nella macchinetta.

Ancora un esempio: vending machine (2)

La macchina all'inizio contiene 10 unita' per ogni bevanda; l'atto di erogazione di una bevanda corrisponde alla diminuzione di un'unita' della disponibilita' della stessa e alla conservazione della moneta (nelle monete conservate, non bisogna distinguere tra monete da 50 centesimi ed 1 euro).

Se la bevanda non e' disponibile, non viene erogata e la moneta non viene conservata.

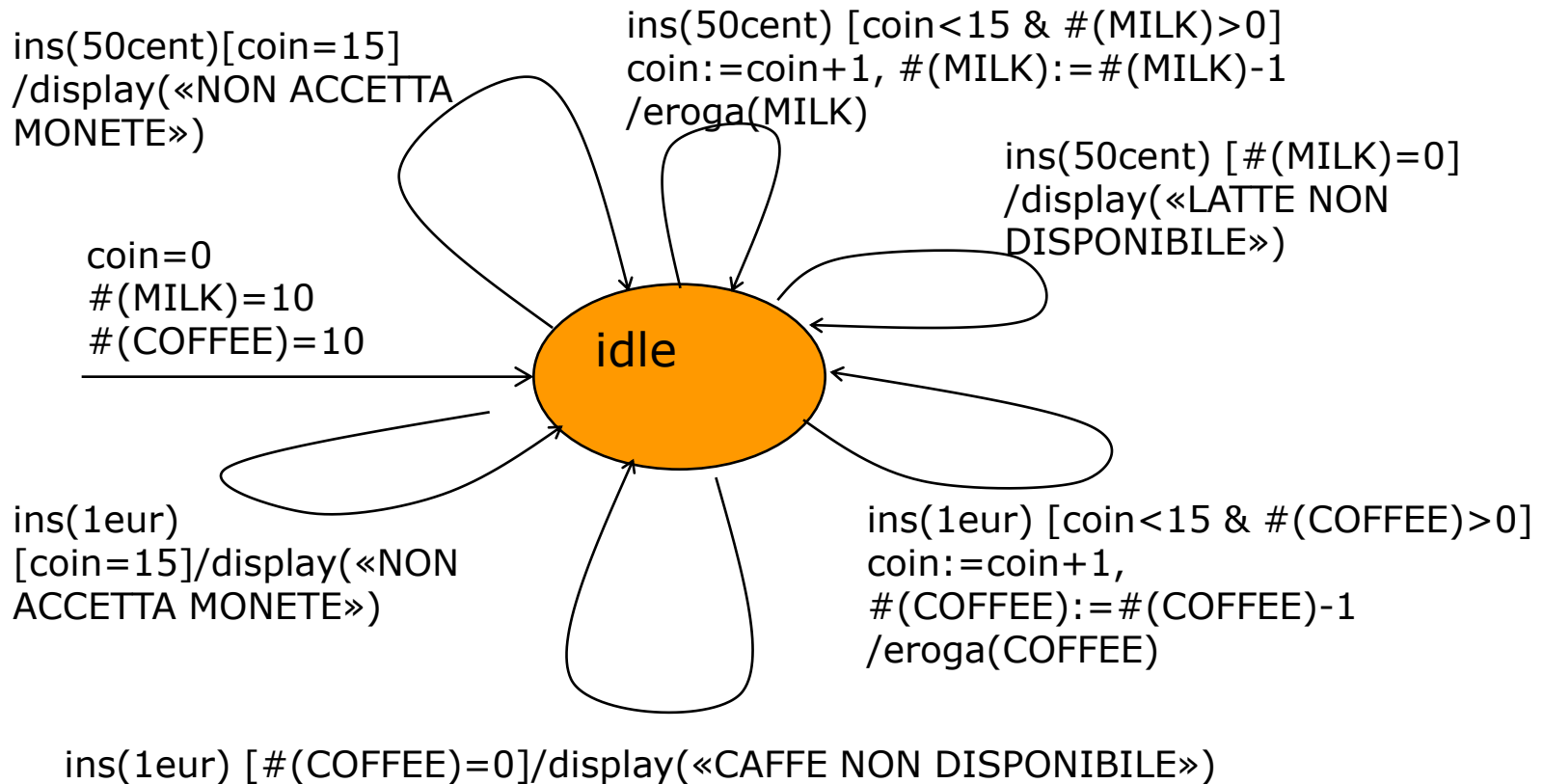
Ancora un esempio: vending machine (3)

La macchina puo' contenere al massimo 15 monete; quando la macchina e' piena di monete, non accetta altre monete e, quindi, non eroga più alcuna bevanda.

All'inizio la macchinetta non contiene alcuna moneta.

L'utente del sistema decide ad ogni passo di simulazione il tipo di moneta da inserire.

Se modelliamo con EFSM



-
- Proviamo adesso a modellare con insiemi, funzioni e regole in ASM.....

vending machine

Segnatura:

domain CoinType = {HALF | ONE}

domain Product = {COFFEE | MILK}

domain QuantityDomain subsetof Integer

domain CoinDomain subsetof Integer

available: Product -> QuantityDomain

coins: CoinDomain

insertedCoin: CoinType

vending machine

definitions:

domain QuantityDomain = $\{0 \dots 10\}$

domain CoinDomain = $\{0 \dots 15\}$

Stato iniziale s0:

coins = 0

available(\$p in Product) = 10

vending machine

```
main rule r_Main =  
    if(coins < 15) then  
        if(insertedCoin = HALF) then  
            if(available(MILK) > 0) then  
                r_serveProduct[MILK]  
            endif  
        else  
            if(available(COFFEE) > 0) then  
                r_serveProduct[COFFEE]  
            endif  
        endif  
    endif
```

vending machine

```
rule r_serveProduct($p in Product) =  
  par  
    available($p) := available($p) - 1  
    coins := coins + 1  
  endpar
```

Verso la formalizzazione....

- Vediamo come si arriva alla definizione formale di:
 - Stato ASM
 - Gli **stati** di una ASM sono delle strutture algebriche, dette (semplicemente) *algebre*
 - Transizione ASM
 - Che permettono di modificare la struttura algebrica durante la *run* o esecuzione della ASM

Vocabolario

- DEF. Un **vocabolario** (o **segnatura**) Σ è una collezione finita di **nomi di funzioni**
- DEF. Fissato un vocabolario Σ , uno **stato** A su Σ è un insieme non vuoto X , detto **superuniverso** di A , con le interpretazioni dei nomi delle funzioni di Σ .
 - Se f è un nome di funzione n -aria di Σ , allora la sua interpretazione f^A è una funzione da X^n a X
 - Se c è un nome di costante di Σ , allora la sua interpretazione c^A è un elemento di X
- Le **funzioni** possono essere *dinamiche* o *statiche*
 - a seconda che l'interpretazione del nome della funzione cambia o no da uno stato al successivo

Costanti

- Funzioni statiche di arietà zero sono dette **costanti**
 - ogni vocabolario ASM contiene sempre le **costanti statiche** *undef* , *True*, *False*
- *Esempi:*
 - I numeri sono costanti numeriche
 - 1,2, ...
 - L'utente può aggiungerne di sue
 - costante `minimoVoto = 18`
 - `DeltaTime= ...`

Funzioni statiche

- Le funzioni statiche (di arietà >0) sono definite tramite una legge fissa
- Esempio di funzioni statiche sono le usuali operazioni
 - tra numeri: $+, -, \dots$
 - tra booleani: AND, OR, ...
- Sono "standard"
- L'utente può definirne di sue
 - es.: $\text{max}(n, m)$

Funzioni dinamiche

- Cambiano la propria interpretazione al variare dello stato
- Funzioni dinamiche di aridità zero sono le comuni variabili dei linguaggi di programmazione
- Es: **insertCoin, DisplayTime**

Esempio di stato ASM

- Modello ASM: CLOCK real time
- Vocabolario (o segnatura) Σ
 - Funzioni dinamiche
 - **CurrTime**: Real (viene incrementato dall'esterno)
 - **DisplayTime**: Real
 - Funzioni statiche
 - **Delta**: Real (intervallo di tempo)
 - **+** : Real x Real $\rightarrow$ Real (addizione)

Domini ASM

Il superuniverso di uno stato ASM è “suddiviso” in *universi*, rappresentati dalle loro funzioni caratteristiche

Se X è il superuniverso dello stato A , l' *universo* G di A è rappresentato dalla funzione $G: X \rightarrow \text{Bool}$ tale che

$$G(t) = \text{true}$$

per tutti gli elementi t *del* superuniverso X di A che formano la partizione (o sotto insieme) G ;

$$G(t) = \text{false, altrimenti}$$

Esempio: $X = \{1, 2, 3, a, b, \text{mario}, \text{pippo}\}$ ripartito in domini:
 $\text{Interi} = \{1, 2, 3\}$, $\text{Char} = \{a, b\}$, $\text{String} = \{\text{mario}, \text{pippo}\}$

Termini ASM

DEF. I *termini* di Σ sono espressioni sintattiche così costruite:

1. Variabili $v_0, v_1, v_2, \dots$ sono termini
2. Costanti c of Σ sono termini
3. Se f è un nome di funzione n -aria di Σ e $t_1, \dots, t_n$ sono termini, allora $f(t_1, \dots, t_n)$ è un termine.

Un termine che non contiene variabili è detto *chiuso*.

Esempi: insertCoin, available(MILK), DisplayTime, ...